1. O. Ω. Θ. o. ω 渐近复杂度.

2. 关于递推的时间复杂度计算，若 $T(n)=\begin{cases} c, & n=1 \\ aT(n/b)+c, & n>1 \end{cases}$

(即一次分治需要分成a个a问题，每个规模为b.
合并起来需要c的时间)

$$T(n)=aT(\tfrac{n}{b})+c$$
$$=a^2T(\tfrac{n}{b^2})+ac+c$$
$$=a^3T(\tfrac{n}{b^3})+a^2c+ac+c$$
$$\cdots$$
$$=a^kT(\tfrac{n}{b^k})+a^{k-1}c+\cdots+ac+c. \quad (k=\log_b n)$$
$$=a^kc+\cdots ac+c$$
$$=c\cdot\frac{a^{k+1}-1}{a-1}$$
$$=c\cdot\frac{an^{\log_b a}-1}{a-1} \quad (a^{\log_b n}=n^{\log_b a})$$

3. 分治法能解决的问题： P40
1) 问题规模缩小到一定程度就很容易解决
2) 问题可以分解成若干规模较小的相同问题
3) 子问题解可以合并为该问题的解.
4) 各子问题相互独立，不重叠.

4. 二分搜索、大整数的乘法（≈Strassen矩阵乘法）、棋盘覆盖、归并排序、快速排序、线性时间选择、最接近点对问题（有没有一下代码都住了什么参数吧，太多了……）、全排列

5. 递归函数的两大要素：边界条件、递归方程.

6. 矩阵连乘问题：① 完全加括号的递归定义
("代价") ② $A[i][j]$ 表示从 A_i 乘到 A_j 在 A_k 处断开所需乘法次数最少
相应再定义一个 $m[i][j]$ 表示计算次数

(计算的顺序比较奇怪，注意一下)

③ $m[i][j]=\begin{cases} 0, & i=j \\ \min\limits_{i\le k<j}\{m[i][k]+m[k+1][j]+p_{i-1}\cdot p_k\cdot p_j\}, & i<j \end{cases}$

(p_i 是 A_i 的列数)

(因为该元素的计算依赖于同行的前面元素和同列的后面元素，只能)

扫描全能王 创建

7. 动态规划的基本要素：最优子结构、重叠子问题

该问题的最优解包含子问题的最优解

☆（证明时假设子问题不是最优解，然后导出原问题的更优解，矛盾）

递归算法求解问题时，每次产生的

子问题并不总是新问题，有些问题被计算多次——子问题的重叠性质

备忘录方法：本质是递归，但是像是动态规划的变形，

以矩阵问题为例，也引入一个m数组，但初始化全为0，然后递归求解m[i][n]，只有遇到非零值或i==j才有返回值，否则一直递归下去，且"这一趟"的递归不光计算了值，还记录在了m[i][j]当中，也相当于记录下了子问题的解

8. 最长公共子序列、凸多边形最优三角剖分、多边形游戏、图像压缩、电路布线、流水作业调度、0-1背包、最优二叉搜索树、最大子段和

9. 贪心算法可解的问题：贪心选择性质 和 最优子结构性质

所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到

（只需证明①贪心选择开始是可行的 ②最优子结构）

10. 活动安排问题、最优装载、哈夫曼编码（证明……）

单源最短路径、最小生成树、多机调度问题（近似）

11. 回溯法：深度优先搜索 + 剪枝

一定记得要回溯！

显约束："明面"上的取值限定；隐约束：问题解所需的约束

扩展结点、活结点、死结点、限界函数

子集树与排列树的算法"模板"

记得更新当前解再进下一层！

12. 装载问题：批作业处理调度、装载三角形问题、

几后问题、最大团问题、图的m着色问题、旅行售货员

（下面这几个是看不懂级别的）圆排列问题、电路板排列~、连续邮资~

基本都会用 x[n] 来记录当前解 bestx[n] 最优解

扫描全能王 创建

13、分支限界法：广度优先 / 最小耗费优先

每个活结点只有一次机会成为扩展结点

无法求出所有解，只能求出一个解或"最优解"

分为"队列式～"和"优先队列式～"

14、单源最短路径、装载问题、布线问题(?)，0-1背包(优先级?)

(检查是否可行、入队列、取下一结点)

最大团问题、旅行售货商

15、判定问题 ⟷ 最优化问题

若最优化问题可在多项式时间求解，则对应判定问题也可

若判定问题不能在多项式时间求解，则对应最优化问题也不能

16、问题变换：问题A变换为问题B指：

1) A的输入变换为B的适当输入

2) 解出问题B，

3) B的输出变换为A的正确解

若用 $O(\tau(n))$ 时间可以完成1)、3)的变换，则称问题A $\tau(n)$ 时间可变换为问题B，记为 $A \propto_{\tau(n)} B$

17、计算时间下界归约：若A的计算时间下界为 $T(n)$，且 $A \propto_{\tau(n)} B$，则 $T(n) - O(\tau(n))$ 是B的一个下界

～上界归约：若B的计算时间上界是 $T(n)$，且 $A \propto_{\tau(n)} B$，则 $T(n) + O(\tau(n))$ 是A的一个上界

e.g.：集合不相交问题 $\propto_{O(n)}$ 排序问题

18. P：在多项式时间内使用确定性算法可解的判定问题的集合

NP：多项式时间内使用非确定性算法可解的所有判定问题的集合

19. P类问题可在多项式时间内解决并验证。

NP类：验证但不确定能否在多项式时间解决

扫描全能王 创建

20. 多项式时间变换 $X \propto_p Y$ 定义为：

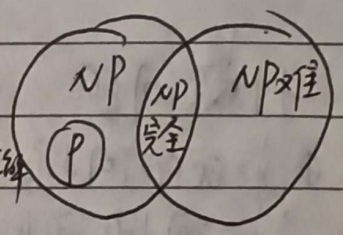
设X和Y是两个分别定义在实例集I和J上的判定问题。

若存在映射 $f: I \to J$，f满足：1) f是多项式时间内可计算的函数

当且仅当？ 2) $x \in X \Leftrightarrow f(x) \in Y$ ；则称X能在多项式时间内变换为问题Y

21. NPC（NP完全）：一个问题L，若① $L \in NP$，且② 对于所有 $L' \in NP$，$L' \propto_p L$，则称L是NP完全的

NP难：只满足性质②的问题L



NPC：可多项式时间验证，

可在多项式时间求解 ⇔ 所有NPC问题可在多项式时间求解

NP难：多项式时间验证都做不到。

22. 定理：设L是NPC，则：

1) $L \in P$ 当且仅当 $P = NP$ （任一NPC问题若可多项式时间求解，则所有都可以）

2) 若 $L \propto_p L_1$，且 $L_1 \in NP$，则 $L_1 \in NPC$

证明NPC.

第一个NPC问题：布尔表达式的可满足性问题SAT

23. 要证明一个问题L是NPC：

1) $L \in NP$，即L的解可以在多项式时间验证。

2) 已知的NPC问题 $L' \propto_p L$ （已知变换为要证）

24. NPC问题：

SAT → CNF-SAT → 3-SAT → 团问题 → 顶点覆盖 → 子集和

3-SAT → 哈密顿圈 → TSP

扫描全能王 创建